

소프트웨어공학

객체지향 설계 구현 테스트

- 1 린바우의 설계기법
- 2 부치의 설계기법
- 3 윌리엄 로렌스의 설계기법
- 4 객체지향 구현
- 5 객체지향테스트



객체지향 설계 기법

객체지향 설계 (OOD: Object Oriented Design)

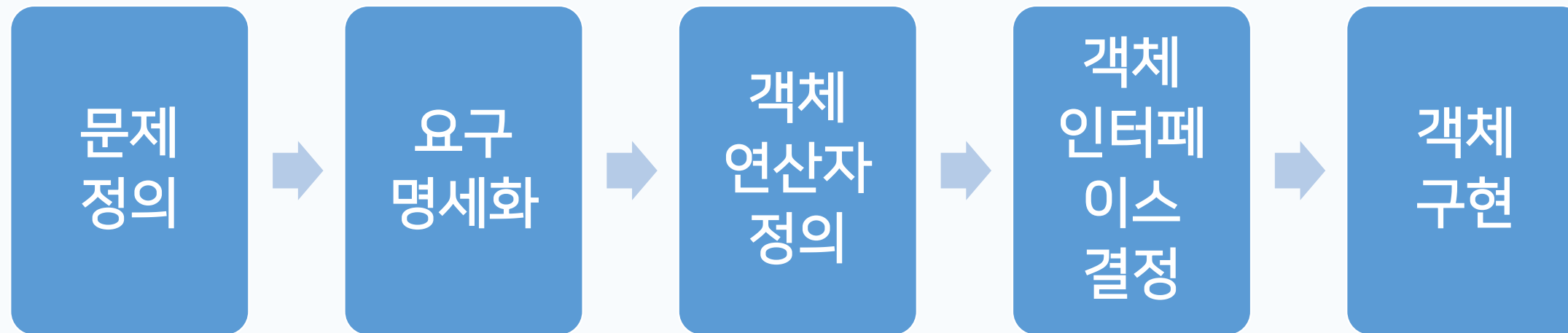
객체지향 분석(OOA)을 사용해서 생성한 여러 가지 분석 모델을 설계 모델로 변환하는 작업으로, 시스템 설계와 객체 설계를 수행한다.

- 최근 소프트웨어 제품의 전형적인 타입인 사용자 중심, 대화식 프로그램의 개발에 적합하다.
- 객체지향 설계에서 가장 중요한 문제는 시스템을 구성하는 객체와 속성, 연산을 인식하는 것이다.
- 객체지향 설계의 설계 개념은 추상화, 정보은닉, 기능 독립성, 모듈화, 상속성을 바탕으로 하며 이 중 가장 중요한 개념은 모듈화이다.
- 객체지향 설계를 위해 럼바우의 객체지향 설계, 부치의 객체지향 설계, 윌리엄 로렌슨(William Lorensen)의 객체지향 설계 방법 등이 제안되었으며, 이 중 일반적으로 럼바우의 객체지향 설계가 가장 많이 사용된다.



객체지향 설계 기법

- 일반적으로 객체지향 설계단계의 순서





객체지향 설계 기법

- 설계원칙:SOLID

SRP(Single Responsibility Principle) 단일 책임 원칙	- 객체는 단 하나의 책임 - 응집도는 높게, 결합도는 낮게 설계
OCP(Open-Closed Principle) 개방-폐쇄 원칙	- 변경에는 닫혀있고, 확장에는 열려있다는 법칙 - 기존의 코드 변경 없이 기능을 추가할 수 있도록 설계 - 공통 인터페이스를 하나의 인터페이스로 묶어 캡슐화 - 디자인 패턴들의 기본 원칙이 되는 원칙으로 변하는 부분과 변하지 않는 부분 분리
LSP(Liskov Substitution Principle) 리스코프 치환 원칙	- 자식(Sub)은 부모(Super)로 치환 가능 - 하위 클래스는 상위 클래스에서 가능한 행위 수행 가능 - 하위 클래스는 상위 클래스를 무시하거나 재정의하지 않고 확장만 수행
ISP(Interface Segregation Principle) 인터페이스 분리 원칙	- 사용하지 않는 인터페이스와 의존 관계를 맺거나 영향을 받지 않아야 함 - 인터페이스가 갖는 하나의 책임
DIP(Dependency Inversion Principle) 의존 역전 원칙	- 상위 모듈은 하위 모듈들에 의존해서 안되며(완벽하게 분리) - 모든 것은 추상에 의존 - 추상성이 낮은 클래스보다 추상성이 높은 클래스와 의존관계를 맺어야 함

01

럼바우의 설계기법

- 럼바우(Rumbaugh)의 객체지향 설계
 - 시스템 설계

시스템 설계

전체적인 시스템 구조를 설계하는 것으로 분석 단계의 분석 모델을 서브시스템으로 분할하고, 시스템의 계층을 정의하며 분할 과정 중에서 성능의 최적 방안, 문제 해결 전략, 자원 분해 등을 확정하는 것이다.

- 상세 설계를 위한 중요한 개념과 전략을 결정하고, 서브 시스템과 이들이 할당될 하드웨어를 결정한다.

럼바우의 설계기법

- 럼바우(Rumbaugh)의 객체지향 설계
 - 시스템 설계 절차
 - ① 시스템을 서브시스템으로 분할한다.
 - ② 동적 모델을 분석하여 객체들의 병행수행 가능성을 파악한다.
 - ③ 서브 시스템을 하드웨어와 태스크에 할당한다.
 - ④ 자원 관리 방법 및 공동 자원의 접근 방법을 결정한다.
 - ⑤ 시스템의 제어 방식을 결정한다.
 - ⑥ 경계 조건의 처리 방법을 결정한다.
 - ⑦ 우선 순위를 결정한다.

01

럼바우의 설계기법

- 럼바우(Rumbaugh)의 객체지향 설계
 - 객체 설계

객체 설계

분석 단계에서 만들어진 클래스, 속성, 관계, 메시지를 이용한 통신들을 설계 모델로 제작하고 상세화하여 구체적인 자료구조와 알고리즘을 정의한다.

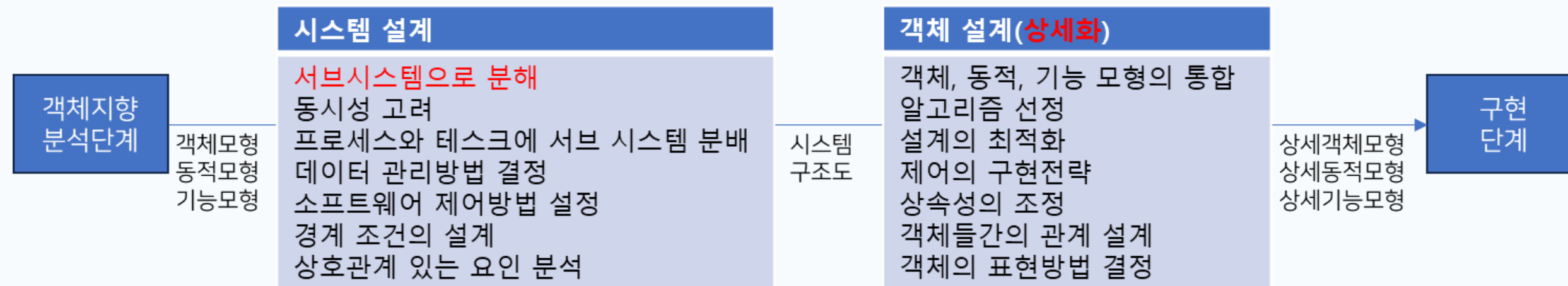
- 정보와 처리를 모듈화하고 데이터 객체와 처리 조작을 연결하며, 추상화, 정보은닉, 모듈화를 기본으로 하여 소프트웨어를 생성한다.

- 럼바우(Rumbaugh)의 객체지향 설계
 - 객체 설계 절차
 - ① 객체 모델링, 동적 모델링, 기능 모델링을 통합하고 연산을 파악한다.
 - ② 연산을 구현하기 위해 알고리즘을 설계한다.
 - ③ 외부와 상호작용하기 위한 제어 방식을 구현한다.
 - ④ 클래스 구조를 조정하여 상속성을 향상시킨다.
 - ⑤ 관계를 설계하고, 객체의 표현 방법을 결정한다.
 - ⑥ 클래스와 관계를 단일 모듈로 생성한다.
 - ⑦ 문서화 한다.

01

럼바우의 설계기법

- 럼바우(Rumbaugh)의 객체지향 설계
 - 절차



부치의 설계기법

- 부치(Booch)의 객체지향 설계

부치의 객체지향 설계	자료 흐름도 (DFD)를 사용해서 객체를 분해하고, 객체들 간의 인터페이스를 찾아 이것들을 Ada 프로그램으로 변환시키는 기법이다.
----------------	---

- 부치 객체지향 설계 절차

- ① 문제 정의(요구사항 분석) 단계
- ② 실세계 문제 영역을 소프트웨어로 구현하기 위한 비정형적인 전략 단계
- ③ 비정형적 전략을 정형화(공식화)하는 단계.
- ④ 위의 1, 2, 3단계를 완전한 설계가 될 때까지 반복한다.
- ⑤ 서브클래스와 메시지 특성을 세분화하여 세부 사항을 정제화한다.
- ⑥ 객체의 속성과 자료 구조를 표현한다.
- ⑦ 구체적인 절차를 표현한다.

- 부치(Booch)의 객체지향 설계
 - ① 문제 정의 단계 → 단일문장으로 기술, 알려진 제약사항 분석
 - 클래스 다이어그램(클래스 간 관계)
 - 객체 다이어그램(객체 간 메시지 표현)
 - 모듈 다이어그램(프로그램 구성요소 표현)
 - 프로세스 다이어그램(처리 표현)
 - ② 비정형적인 전략 단계
 - 직설적 문장으로 표현, 동일 추상화 수준 기술
 - 절차적 상세화가 아닌 문제 해결에 중점

02

부치의 설계기법

- 부치(Booch)의 객체지향 설계
 - ③ 정형화(공식화) 단계
 - 객체, 속성, 연산 식별
 - 구성요소와 인터페이스 식별
 - 도식적 표현
 - 구현의 상세화

윌리엄 로렌스의 설계기법

윌리엄 로렌스의 객체지향 설계

추상화, 상속성, 메시지, 그리고 다른 모든 OOD 개념들을 직접 지원해 주는 기능을 갖추고 있는 Smaltalk와 같은 프로그래밍 언어로 소프트웨어를 개발하기 위한 기법이다.

- 윌리엄 로렌스 객체지향 설계 절차

- ① 각 서브시스템에 대한 자료 추상화를 식별한다.
- ② 각 추상화에 대한 속성들을 식별한다.
- ③ 각 추상화에 대한 연산들을 식별한다
- ④ 객체들 사이의 통신 메시지를 식별한다.
- ⑤ 시나리오를 이용하여 설계를 검사한다.
- ⑥ 적절한 곳에 상속을 적용한다.

04

객체지향 구현

객체지향 구현

구현은 설계 단계에서 생성된 설계 모델과 명세서를 근거로 하여 코딩하는 단계이다.

- 객체지향 프로그래밍을 이용하면 용이하게 구현할 수 있다.
- 객체는 순차적으로(sequentially) 또는 동시적으로(concurrently) 구현될 수 있다.

04

객체지향 구현

- 객체지향 프로그래밍(OOP: Object Oriented Programming)

객체지향 프로그래밍

새로운 개념의 모듈단위, 즉 객체라는 단위를 중심으로 하여 프로그램을 개발하는 기법이다.

- 객체라는 단위를 이용하여 현실 세계에 가까운 방식으로 프로그래밍 한다.
- 현실 세계에 가까운 방식이므로 이해하기 쉽고 조작하기 쉬운 프로그램을 개발할 수 있다.
- 유지보수가 쉽고 재사용 가능한 프로그램을 만들 수 있다.
- 이미 개발된 프로그램을 이용해 빠르게 확장된 프로그램을 개발 할 수 있다.
- 대표적인 객체지향 프로그래밍 언어로는 Smalltalk, C++, JAVA등이 있다.

04

객체지향 구현

- 객체지향 프로그래밍의 특징
 - 재사용성(생산성 향상)
 - 확장 용이성(새로운 기능, 객체 추가 용이)
 - 신뢰성(오류없는 정확한 결과)
- 객체지향 : 자료중심
 - → data+behavior
 - data: 자료(속성)
 - behavior: 행위, 연산(메서드), sub program 절차
 - 객체의 집합 → 클래스
 - 호출에 사용되는 데이터 → 메시지
- 객체의 속성: 유일성(식별성), 분류성(집단화), 상속성, 다형성(다양한 기능)

객체지향 테스트

클래스 테스트 (단위 테스트)

구조적 기법에서의 단위 테스트와 같은 개념으로 캡슐화된 클래스나 객체를 검사하는 것이다.

- **통합 테스트** : 객체를 몇 개 결합하여 하나의 시스템으로 완성시키는 과정에서의 검사로, 스레드 기반 테스트와 사용 기반 테스트로 분류할 수 있다.

스레드 기반 (thread-based) 테스트

시스템에 대한 하나의 입력이나 이벤트에 응답하는 데 요구되는 클래스들을 통합하는 것으로, 각각의 스레드가 통합되고 개별적으로 테스트 된다.

사용 기반 (use-based) 테스트

독립 클래스를 테스트한 후 독립 클래스를 사용하는 다음 계층의 종속 클래스를 테스트한다.

- **확인(검증) 테스트** : 사용자 요구사항에 대한 만족 여부를 검사한다.
- **시스템 테스트** : 모든 요소들이 적합하게 통합되고 올바른 기능을 수행하는지 검사한다.

05

객체지향 테스트

- 객체지향 테스트 단계.

